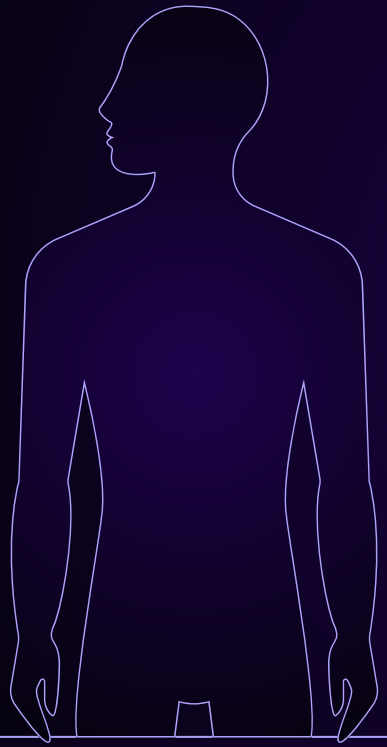
Klaudia KUBALE
클라우디아 쿠바라
24170166



BONE FRACTURE DETECTION

Computer Vision term project

WHY THIS SUBJECT?

- Need of an accurate and rapid diagnostic
- Interesting in resource limited work places
- Automation to help reducing workload, prevent mistakes

SUMMARY

Data & Preprocessing

Model & Training

Results

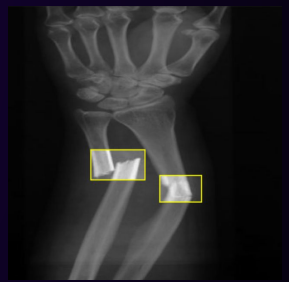
Demo

DATASET AND PREPROCESSING

- Alvusion Fracture
- Comminuted Fracture
- Compression-Crush Fracture
- Fracture Dislocation
- GreenStick Fracture
- HairLine Fracture
- Impact Fracture

A small piece of bone attached to a tendon gets pulled away.

Bones are forced out of their normal position



Label verification: Keep only high-quality annotation, no noisy data to prevent the model performance to drop.

Cleaning: Remove small classes, outliers to prevent label noise.

Dataset structure: 3 sets train for model learning, valid for tuning and monitoring during training, and test for final evaluation.

https://universe.roboflow.com/ahmedali-aqmwq/break-bone

DATASET AND PREPROCESSING

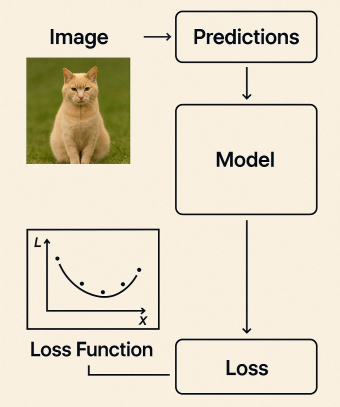
```python
classes_to_remove = [7, 8, 9, 10]  # null, oblique, spiral, Intra-articular

splits = ["train", "valid", "test"]
base_dir = "Break-bone-3"

for split in splits:
    labels_dir = os.path.join(base_dir, split, "labels")
    images_dir = os.path.join(base_dir, split, "images")
    for fname in os.listdir(labels_dir):
        if fname.endswith(".txt"):
            label_path = os.path.join(labels_dir, fname)
            with open(label_path, "r") as f:
                lines = f.readlines()
            to_delete = any(line.strip() and int(line.split()[0]) in classes_to_remove for line in lines)
            if to_delete:
                os.remove(label_path)
                base_name = os.path.splitext(fname)[0]
                for ext in [".jpg", ".jpeg", ".png"]:
                    img_path = os.path.join(images_dir, base_name + ext)
                    if os.path.exists(img_path):
                        os.remove(img_path)
```

MODEL AND TRAINING

ultralytics
YOLOv8



- Image as input
- The model do prediction
- Predicts bounding boxes and class labels → forward pass
- Backpropagation and adjusting value

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
model.train(data='./Break-bone-3/data.yaml', epochs=70, imgsz=640, device='cpu', batch=8)
```

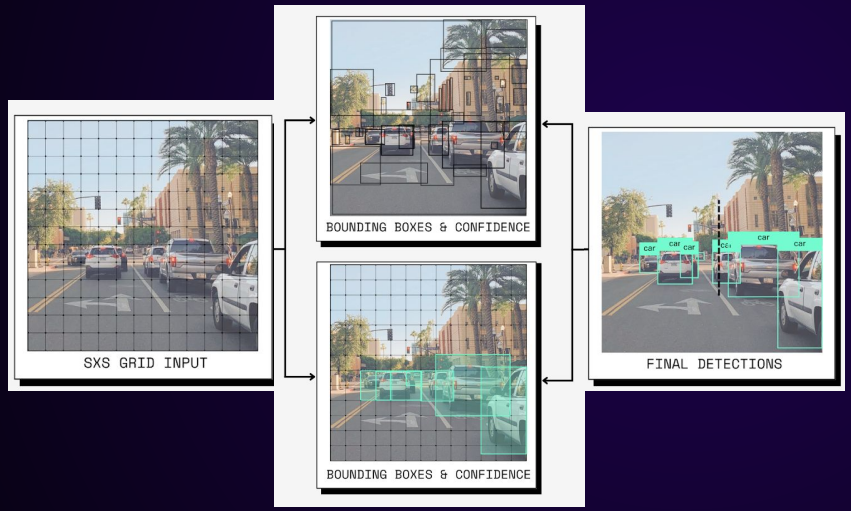
Complete pass through the entire training dataset + update the weights based on the loss, so it improve the performance

A batch subset of the training data processed together

Small batch = more updates, less memory, possibly better generalization. Large batch = more memory, faster per-epoch training.

AFTER THE TRAINING

For inference, YOLO uses the single pass process:



- Divides the image into a grid
- Predicts bounding boxes and class labels

MODEL AND TRAINING

The precision at the start of the training

```
ting training for 70 epochs...

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
   1/70         0G      2.702      5.599      2.261         15        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 10
                all        361        512      0.465     0.0193    0.00765    0.00244

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
   2/70         0G      2.549      4.976      2.124         10        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 10
                all        361        512      0.477     0.0298     0.0215     0.0054

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
   3/70         0G      2.543      4.563      2.104         11        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 10
                all        361        512      0.094     0.0788     0.0279    0.00897

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
   4/70         0G      2.575       4.38      2.134         13        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 10
                all        361        512     0.0082      0.434     0.0328    0.00965

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
   5/70         0G      2.506      4.003      2.099         16        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 10
                all        361        512      0.226      0.135      0.065     0.0215
```

```
  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  65/70         0G      1.616      1.199      1.586          9        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                all        361        512      0.731      0.698       0.72      0.285

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  66/70         0G      1.607       1.19      1.572          8        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                all        361        512       0.77      0.689      0.722      0.283

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  67/70         0G      1.611      1.204      1.579          8        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                all        361        512      0.749      0.688      0.724      0.281

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  68/70         0G      1.595      1.193      1.581          9        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                all        361        512      0.744      0.683      0.712      0.279

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  69/70         0G      1.595      1.154      1.574          6        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                all        361        512      0.744      0.689      0.717      0.284

  Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
  70/70         0G      1.585      1.183      1.571         10        640: 100%|
              Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                all        361        512      0.729      0.705      0.711      0.283
```

At the end

RESULTS

```
Optimizer stripped from runs\detect\train9\weights\last.pt, 6.3MB
Optimizer stripped from runs\detect\train9\weights\best.pt, 6.3MB

Validating runs\detect\train9\weights\best.pt...
Ultralytics 8.3.38 🚀 Python-3.12.10 torch-2.5.1+cu118 CPU (Intel Core(TM) i5-10300H 2.50GHz)
Model summary (fused): 168 layers, 3,007,013 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 23/23 [00:39<00:00,  1.73s/it]
                   all        361        512      0.761      0.686      0.721      0.284
     Avlusion Fracture         46         57       0.86      0.807      0.854      0.333
   Comminuted Fracture         87        137      0.755      0.767      0.729      0.277
Compression-Crush Fracutre         63         82      0.753      0.768      0.776      0.398
  Fracture Dislocation         57         79      0.612      0.678      0.614      0.211
   GreenStick Fracture         37         66      0.786      0.576      0.685      0.244
     HairLine Fracture         31         44      0.847      0.631      0.734       0.26
       Impact Fracture         39         47       0.71      0.573      0.655      0.267
Speed: 2.7ms preprocess, 98.1ms inference, 0.0ms loss, 0.5ms postprocess per image
Results saved to runs\detect\train9
```

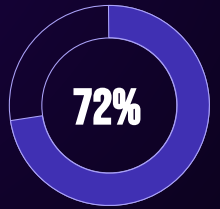
Average Precision (AP): AP computes the area under the precision-recall curve

After 70 epochs:

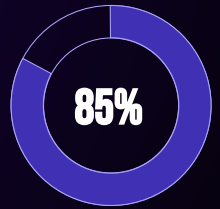
- the mean AP is at 0.72
- Good performance especially for avulsion fractures → 0.85
- if we keep training >80%?

ACCURACY

ALL CLASSES



AVULSION



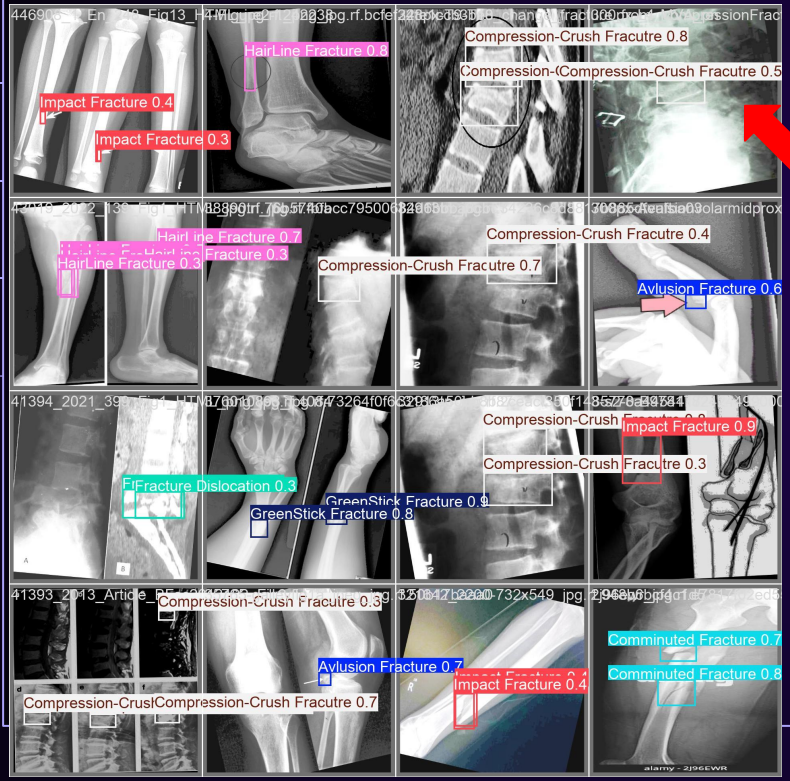
RESULTS

Class	mAP@0.5	mAP@0.5:0.95	Notes
Avulsion Fracture	0.854	0.333	✅ Excellent detection
Comminuted Fracture	0.729	0.277	✅ Solid performance
Compression-Crush	0.776	0.398	✅ Highest score!
Hairline Fracture	0.731	0.260	👍 Good
Impact Fracture	0.655	0.267	👍 Decent
Fracture Dislocation	0.614	0.211	🔸 Could improve
GreenStick Fracture	0.685	0.244	🔸 Stable, decent

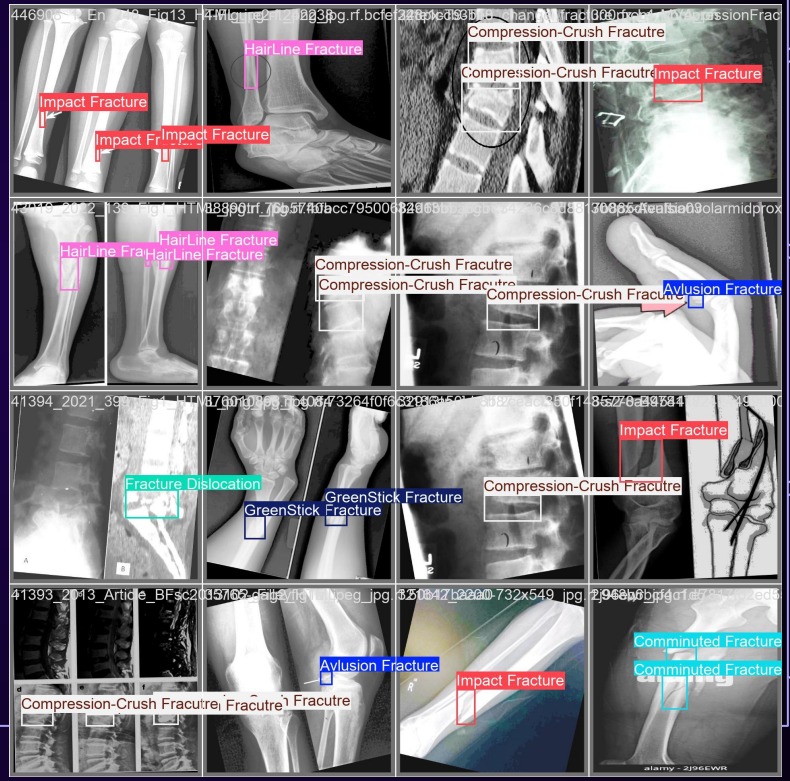
The mAP@0.5:0.95 score is lower, as it requires more precise localization

RESULTS

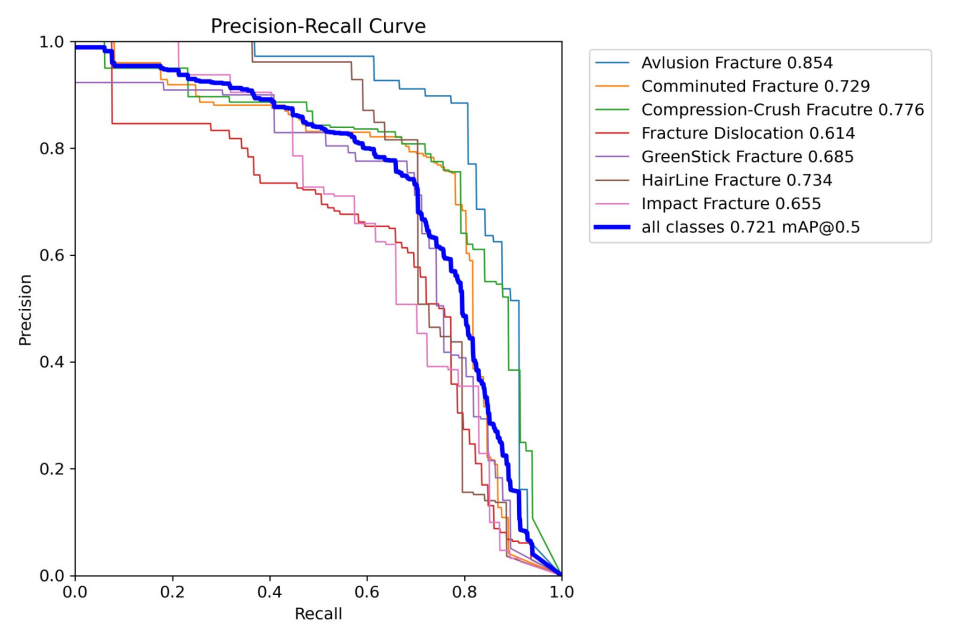
PREDICTION

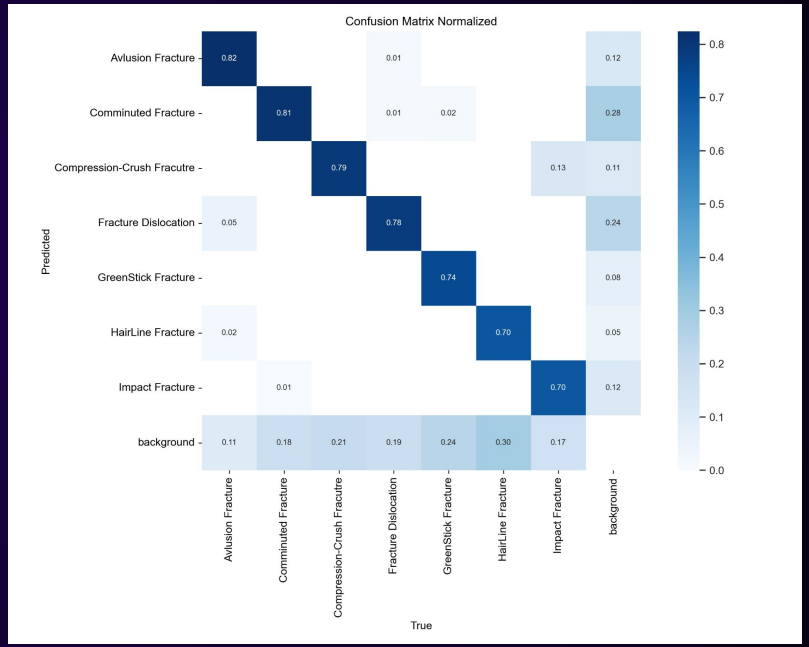


LABELED

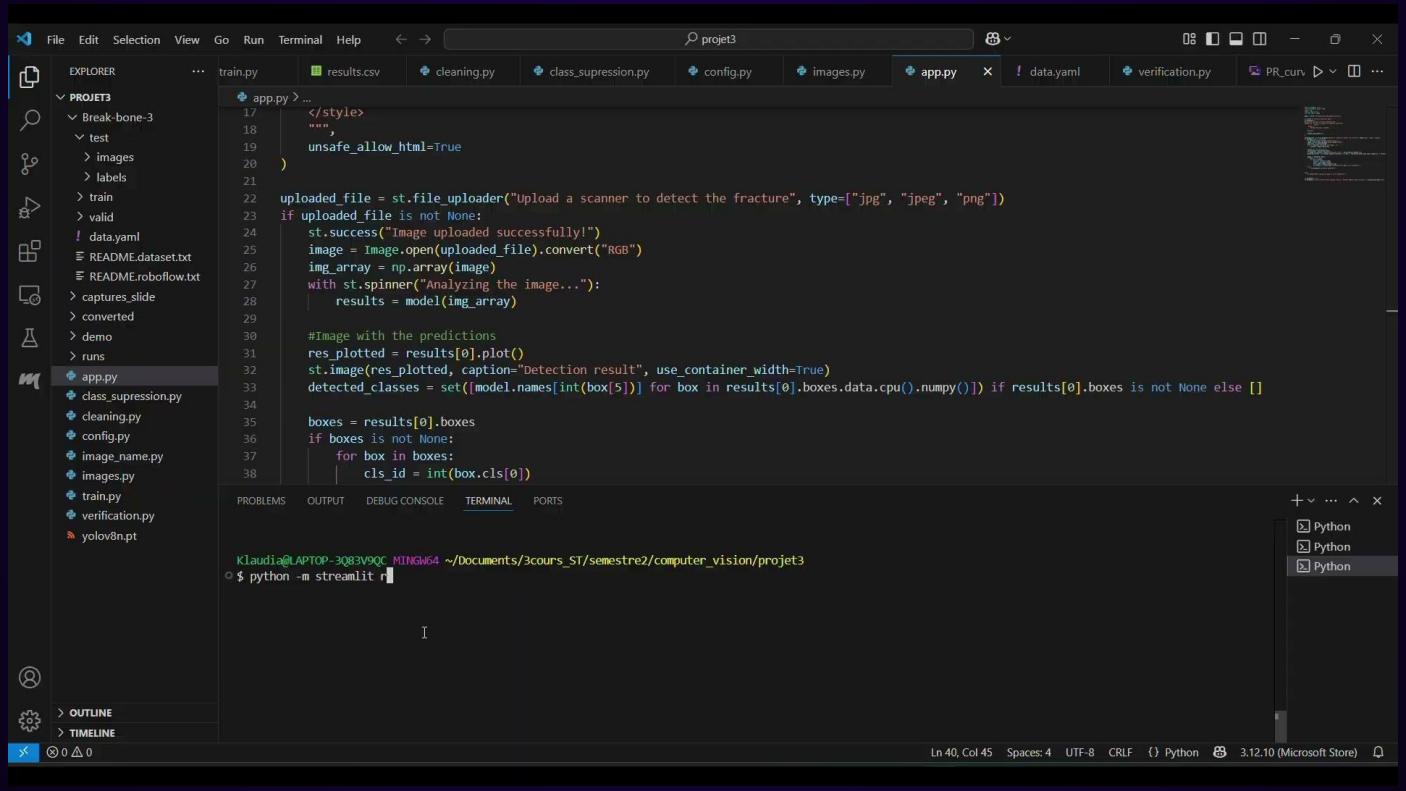


RESULTS


Precision-Recall Curve
Precision
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Avlusion Fracture 0.854
Comminuted Fracture 0.729
Compression-Crush Fracutre 0.776
Fracture Dislocation 0.614
GreenStick Fracture 0.685
HairLine Fracture 0.734
Impact Fracture 0.655
all classes 0.721 mAP@0.5


Confusion Matrix Normalized
Predicted
True
Avlusion Fracture
Comminuted Fracture
Compression-Crush Fracutre
Fracture Dislocation
GreenStick Fracture
HairLine Fracture
Impact Fracture
background
0.82 0.01 0.12
0.81 0.01 0.02 0.28
0.79 0.13 0.11
0.05 0.78 0.24
0.74 0.08
0.02 0.70 0.05
0.01 0.70 0.12
0.11 0.18 0.21 0.19 0.24 0.30 0.17
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0

DEMO: APP FOR AUTOMATIC DETECTION



DEMO: APP FOR AUTOMATIC DETECTION

Image pretreatment

Image and result display

```python
uploaded_file = st.file_uploader("Upload a scanner to detect the fracture", type=["jpg", "jpeg", "png"])
if uploaded_file is not None:
    st.success("Image uploaded successfully!")
    image = Image.open(uploaded_file).convert("RGB")
    image = image.resize((640, 640))
    img_array = np.array(image)
    gray = cv2.cvtColor(img_array, cv2.COLOR_RGB2GRAY)
    equalized = cv2.equalizeHist(gray)
    img_array = cv2.cvtColor(equalized, cv2.COLOR_GRAY2RGB)
    with st.spinner("Analyzing the image..."):
        results = model(img_array)

    #Image with the predictions
    res_plotted = results[0].plot()
    st.image(res_plotted, caption="Detection result", use_container_width=True)
    detected_classes = set([model.names[int(box[5])] for box in results[0].boxes.data.cpu().numpy()]) if results[0].boxes is not None else []

    boxes = results[0].boxes
    if boxes is not None:
        for box in boxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            class_name = model.names[cls_id]
            st.write(f" •  {class_name} detected with {conf:.2%} confidence.")
    else:
        st.warning("No fracture detected.")


else:
    st.info("Please upload an image to start detection.")
```

CONCLUSION

After cleaning process, utilizing the YOLOv8 model, the mean Average Precision (mAP) achieved 0.72 for all classes after 70 epochs of training. Notably, the model showed excellent performance for Avulsion Fractures, with an mAP of 0.85.

The insights from our precision-recall curve and confusion matrix highlight areas of strength and also guide for future improvements. For example, what type of data we need to perfect the model, particularly for certain fracture types. In summary, the results clearly show the viability and accuracy of an automated solution for aiding medical professionals.

Thank you for your attention